Regis University CPS
CS361 Programming Assignment 3
due by midnight Sunday of week 3

WARNING: The programs this week will take MUCH longer to complete than the previous 2 weeks assignments. Be sure to take this into account when allocating your time.

Program #1

The local yogurt store would like you to write a program to help them calculate the cost of yogurt purchases.

Specifications:

- All yogurt in the store runs $0.44 per ounce, before toppings.
  And all purchases will be in whole ounces.
- Single servings of yogurt may be purchased in a plastic cup or waffle cone.
  - Plastic cups are free.
  - Waffle cones cost an additional $0.57.
- Customers may optionally add toppings to their yogurt at $0.63 per topping.
- Yogurt may be purchased in larger take home amounts for a discount.
  - If customer purchases between 16 and 31 ounces, a $0.72 discount is given.
  - If customer purchases 32 ounces or more, a $1.58 discount is given.
- Punch cards are given out to customer. Every time the customer gets 5 punches on their card, their next purchase is half price (after discounts are applied).

Implementation Requirements:

Data Storage:

- The program must use **constants** for ALL fixed values (therefore, there should be no hardcoded numbers in your program).
- The program shall use descriptive names for all constants and variables, and follow the naming conventions in the Coding Standards.
- The program shall use **double** type constants and variables for all floating point values (***instead of*** **float**).

Required User-Defined Function:

- Create separate **a user-defined function** to calculate the discount.
- The function will take one input parameter: the number of ounces of yogurt
- The function will return the correct discount to give (or 0, if no discount is applicable).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

The program should read all inputs, then perform all calculations, then display all outputs.

The user will enter the following input:

- The number of ounces of yogurt purchased
- Whether the customer wants a plastic cup or a waffle cone
    - Read the answer from the user ('p'/'P' for plastic cup, 'w'/'W' for waffle cone, in either upper or lowercase) and store it in a character variable.
    - Use a pre-defined function from the **cctype** library to convert the stored character to uppercase, for use later when deciding what to charge and what to display.

  NOTE: If the customer orders too many ounces to fit in the waffle cone, and the customer wants a waffle cone, the waffle cone will be placed on top of the yogurt.
- Whether the customer wants to add toppings
    - Read the answer from the user ('t'/'T' for toppings, 'n'/'N' for no toppings, in either upper or lowercase) and store it in a character variable.
    - After reading the input character, the program shall assign a value to a Boolean variable called **wantsToppings** (***true*** for a customer who wants toppings, ***false*** for a customer who does not). This time, the program must use a **logical operator** to test for either upper or lowercase when deciding what value to assign to the Boolean variable (***not*** the **cctype** library function)
    - If the customer wants toppings, also read the number of toppings to add
- Whether the customer has 5 punches on the card
    - Read the answer from the user as a **string**: "yes" or "no"
    - The prompt should indicate that the string must be entered in all lowercase letters.

  NOTE: No error checking of the user input is required. You may assume that the user entered a valid answer for each question, and assume also valid input for test cases.

Sample Input 1:

```
Yogurt Purchase Calculation Program

Enter the weight of the yogurt, in ounces:  22

Plastic cup or waffle cone (P/W)? P

Toppings wanted (T for toppings/N for no toppings)? T
Number of toppings? 2

Are there 5 punches on card (yes or no, all lowercase)? yes
```

Notes on Sample Output:

a) Display a couple of blank lines **after** all input has been entered and **before** the order details and bill are displayed.

b) All order details and bill amounts should line up on the right.

c) The container type should be displayed in a ***word*** (not a character).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

d) Only display a cone cost, if the waffle cone is chosen

d) If the number of toppings is 0, display 0 for the number and 0.00 for the cost.

e) Only display the discount if the discount is greater than 0. If there is no discount, then the subtotal separating line, subtotal, and discount amount will not be displayed at all – just the final separating line and the total.

f) Only display the 5 punch reduced price if it is applicable.

Sample Output 1:

```
YOGURT ORDER
Weight (oz):                   22
Container:             plastic cup
Number of Toppings:             2

YOGURT BILL
Yogurt Cost:                 9.68
Plus Toppings:               1.26
                          --------
Subtotal:                   10.94
Less Discount:               0.72
                          --------
Bill Total:                 10.22

5 punch reduced price:       5.11
```

Sample Input/Output 2:

```
Yogurt Purchase Calculation Program

Enter the weight of the yogurt, in ounces:  10

Plastic cup or waffle cone (P/W)? w

Toppings wanted (T for toppings/N for no toppings)? n

Are there 5 punches on card (yes or no, all lowercase)? no


YOGURT ORDER
Weight (oz):                   10
Container:             waffle cone
Number of Toppings:             0

YOGURT BILL
Yogurt Cost:                 4.40
Plus Cone:                   0.57
Plus Toppings:               0.00
                          --------
Bill Total:                  4.97
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program #2

The IRS allows a taxpayer to contribute to a ROTH IRA, if his/her income is not over a specified limit and his/her filing status allows contributions.

You will write a program that carries out the logic of a simplified worksheet to determine how much may be contributed to a ROTH IRA.

Sample IRS-like instructions for each line of the worksheet:

This worksheet may not be used by any taxpayer who is ***married filing separately***, and did ***not*** live with their spouse any time during the year.	
Line 1	Determine maximum possible ROTH IRA contribution limit. If under age 50, enter $5,500 on line 1. Otherwise, enter $6,500 on line 1.
Line 2	Enter the modified Adjusted Gross Income for IRA purposes on line 2.
Line 3	Determine the Income Limit amount for full contributions, according the filing status from the list below and enter limit on line 3: Single or Head of household - $ 112,000 Married filing jointly or Qualifying Widow(er) - $ 178,000 Married filing separately and lived with spouse any time during year – $ 0
Line 4	Determine if a reduction to the contribution limit is necessary. Is the amount on line 2 less than or equal to the amount on line 3? **Yes**. Can contribute up to maximum contribution limit. Skip lines 4 – 8 and enter the amount from line 1 on line 9 (ignore line 9 instructions) **No**. Contribution limit may be reduced. Subtract line 3 from line 2 and enter the amount on line 4 and continue.
Line 5	Determine division amount to use, depending on filing status, and enter it on line 5: Single or Head of household - $ 15,000 Married filing jointly or separately, or Qualifying Widow(er) – $ 10,000
Line 6	Determine reduction percentage. Divide line 4 by line 5 and determine the floating point result rounded to 3 places. Is the result is 1.0 or more? **Yes**. Cannot contribute to a ROTH IRA. Skip lines 6 – 8 and enter 0 on line 9 (ignore line 9 instructions) **No**. Contribution limit will be reduced. Enter the calculated reduction percentage rounded to 3 places on line 6.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Line 7	Determine reduction amount. Multiply line 6 by line 1 and enter the result on line 7
Line 8	Determine possible contribution limit. Subtract line 7 from line 1. Round the result up to the nearest $10, and enter result on line 8
Line 9	Enter the larger value: line 8 or $200. This line 9 amount is your **ROTH IRA contribution limit**.

The user inputs will be:

- Filing status (one of the following)
  1. Single,
  2. Married, Filing Jointly,
  3. Head of Household,
  4. Qualifying Widow,
  5. Married, Filing Separately.
- If filing status is 5, whether taxpayer lived with spouse at any time during year.
- Age
- Adjusted Gross Income for IRA purposes

Note that when the user enters 1, 2, 3 or 4, you will just read the age and adjusted gross, and not ask about the living situation.

Sample Input 1

```
ROTH IRA Contribution Limit Worksheet Program

1 - Single
2 - Married, Filing Jointly
3 - Head of Household
4 - Qualifying Widow(er)
5 - Married, Filing Separately
Enter your filing status from above (1-5): 2

Enter your age: 45

Enter Adjusted Gross Income for IRA purposes: 180330
```

NOTE: No error checking of any user input is required. You may assume that the user entered a valid answer for each question.

Program Logic Summary

- Read the filing status of the taxpayer.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

- Determine if the taxpayer should continue, or if the program should exit.
    - If the taxpayer's filing status is ***married filing separately***
        - Ask if the taxpayer lived with his/her spouse at any time during the year.

          (User input can be in either upper or lowercase.)
            - If the answer is no, the taxpayer may not contribute to a ROTH IRA. contribution.
            - Display a message saying so (see Sample 2 Output below) and pause.
            - Then return from **main** with exit code of 1, to indicate that the program returned before line 1.
                - `return 1;`
    - If the program did not exit, continue.
- Read and store the taxpayer's age.
- Perform a test to determine and store the amount for line 1.
- Read and store Adjusted Gross Income for IRA purposes for line 2.
- Determine the line 3 income limit that applies for the filing status entered, and store the result.
    - HINT: Store the filing status ***description in words*** into a string variable at the same time.
- Display the Worksheet title and the first 3 lines (1 – 3) of the worksheet.
- Perform the line 4 test to determine if the program needs to continue.
    - If line 2 is less than or equal to line 3
        - Display the amount from line 1 on line 9, and pause.
        - Then return from **main** with exit code of 4, to indicate that the program exited on line 4.
            - `return 4;`
    - If the program did not exit, continue.
- Perform the necessary calculation for line 4 and store the result.
- Determine the line 5 division amount to use for the filing status entered, and store it.
- Display lines (4 and 5) of the worksheet.
- Perform the necessary calculation for line 6 and store the result.

  Perform the line 6 test to determine if the program needs to continue.

  If the result is 1.0 or more, taxpayer cannot contribute to a ROTH IRA:
    - Display a message, and pause (see Sample 4 output)
    - Then return from **main** with exit code of 4, to indicate that the program exited on line 6.
        - `return 6;`
    - If the program did not exit, continue.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

- Perform the necessary calculation for line 7 and store the result.
- Call a **user-defined function** to find the amount for line 8 and store the value returned.
  - ✓ The user-defined function will have two input arguments:
    the **line 1 amount** and the **line 7 amount**
  - ✓ The function will subtract line 7 from line 1.
  - ✓ The function will round the result to the nearest $10.
    Hint: Divide by 10, apply the pre-defined **ceil** function to the resulting value, and then multiply that by 10.
  - ✓ The function will return the result.
  - ✓ Examples:
    - Input arguments of line 7 = 475 and line 1 = 5500
      would return 5030
    - Input arguments of line 7 = 500 and line 1 = 6500
      would return 6000
  - ✓ No other computations should be performed within this function.
- Perform the line 9 test to determine the final contribution limit and store the result.
- Display lines 6 through 9 of the worksheet, making sure that line 6 is displayed to 3 decimal places (see Sample 1 Output below).

Sample 1 Output (no early exit – uses sample inputs 1 from program description)

```
    ROTH IRA Contribution Limit Worksheet

    Filing Status:                          Married, Filing Jointly

1.  Possible contribution limit:             5500.00

2.  Adjusted Gross Income                                  180330.00
3.  Income Limit:                                          178000.00
4.  Amount over Limit:                                       2330.00

5.  Division amount:                        10000.00
6.  Reduction Percentage:                     0.233
7.  Reduction Amount:                        1281.50
8.  Reduced contribution limit:              4220.00

9.  Maximum ROTH IRA Contribution Limit                      4220.00
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample 2 Input

```
ROTH IRA Contribution Limit Worksheet Program

1 - Single
2 - Married, Filing Jointly
3 - Head of Household
4 - Qualifying Widow(er)
5 - Married, Filing Separately
Enter your filing status from above (1-5): 5

Did you live with your spouse for any part of the year (Y/N)? n
```

Sample 2 Output (early exit before line 1)

```
Contributions to a ROTH IRA are not allowed for taxpayers who:
   -  File as Married Filing Separately
   -  Did not live with their spouse for any part of the year.

Therefore, you cannot contribute to a ROTH IRA!
```

Sample 3 Input

```
ROTH IRA Contribution Limit Worksheet Program

1 - Single
2 - Married, Filing Jointly
3 - Head of Household
4 - Qualifying Widow(er)
5 - Married, Filing Separately
Enter your filing status from above (1-5): 1

Enter your age: 65

Enter Adjusted Gross Income for IRA purposes: 66000
```

Sample 3 Output (exits line 4, skips lines 4 through 8)

```
    ROTH IRA Contribution Limit Worksheet

    Filing Status:                                        Single

1.  Possible contribution limit:           6500.00

2.  Adjusted Gross Income                                66000.00
3.  Income Limit:                                       112000.00

9.  Maximum ROTH IRA Contribution Limit                   6500.00
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample 4 Input

```
ROTH IRA Contribution Limit Worksheet Program

1 - Single
2 - Married, Filing Jointly
3 - Head of Household
4 - Qualifying Widow(er)
5 - Married, Filing Separately
Enter your filing status from above (1-5): 3

Enter your age: 35

Enter Adjusted Gross Income for IRA purposes: 128000
```

Sample 4 Output (exits line 6)

```
    ROTH IRA Contribution Limit Worksheet

    Filing Status:                              Head of Household

1.  Possible contribution limit:            5500.00

2.  Adjusted Gross Income:                                 128000.00
3.  Income Limit:                                          112000.00
4.  Amount over Limit:                                      16000.00

5.  Division amount:                       15000.00

    Income too high - You cannot contribute to a ROTH IRA!
```

Sample Output Notes

- Make sure there are a couple of blank lines AFTER all input has been entered and BEFORE the worksheet is displayed.
- The filing status should be displayed in ***words*** (i.e. not a number), lined up on the right with the figures below it.
- A married filing separately filing status may cause the program to exit without displaying any of the worksheet (see sample 2 above).
- The decision on line 4 may cause skipping the display of lines 4 - 8 (see sample 3 above).
- The decision on line 6 may cause an exit after line 5 (see sample 4 above).
- If neither of the above happens, all lines of the worksheet will be displayed (see sample 1 above).

Hints

- First work out the logic in pseudo code, on paper.
- Remember to declare constants for ***all*** fixed values in the program.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

- Use separate variables with descriptive names to store your calculations for each line.
- Work on one line at a time. Make sure each line is working correctly before moving on to the next line and any subsequent calculations.
- Use the sample inputs and outputs to help you test your program.

Documentation

1) **Top of program** comments must include:
   Your name, the course, and the assignment number
   What the program does including:
      What is Input
      Processing Performed
      What is Output

2) Add comment for any constants/variables whose names are not completely descriptive.

3) Add a comment above ***each*** **user-defined function** that includes:
   What the function does
   The name and a description of the parameters that are input to the function
   A name (if available) and a description of what is returned from the function

   Example:

```
// -------------------------------------------------------
// Description: Function sums two numbers
// Input parameters: num1 – first operand
//                   num2 – second operand
// Returns:  sum - sum of num1 and num2
// -------------------------------------------------------
int SumTwo (int num1, int num2)
```

Submission

This programming assignment is due by midnight Sunday. Submit your program source code to the ***Prog Assn 3*** folder in the ***Dropbox*** area of the online course. **Both** program files should be attached to **one** submission for week 3.

Before submitting your program files, you MUST name them as follows:

```
Lastname-wk3-prog1.cpp
Lastname-wk3-prog2.cpp
```

For example:

```
Smith-wk3-prog1.cpp
Smith-wk3-prog2.cpp
```

Grading

The rubric that will be used to grade your program is included on the next pages.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 3 Rubric

Program 1: (48 pts)				
Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Data Storage Points	Constants defined for all fixed values. All variables defined locally, using appropriate data types. Following coding standards naming conventions for all identifiers.	Minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed values OR defined one global variable.	Major errors OR majority of names did not follow naming conventions OR missing multiple constants for fixed values OR defined multiple global variables.	Constants not used, identifier names are not descriptive, and there is a multitude of errors.
Program Input	Displays a descriptive prompt for each input and reads the required input correctly. Accepts one char input in either upper or lowercase, and converts to uppercase using pre-defined function.	Reads required input correctly, but does not display a specific prompt OR reads data using wrong data type OR does not accept and convert either upper or lowercase letters.	Reads some of the required input incorrectly, OR does not read some required input at all.	Hardcoded values used instead of reading input, OR program fails completely.
Program Logic: if statements and calculations	Logic accomplishes all program requirements, including correct math computations. All if statements written correctly. Uses logical operator to convert input value to boolean value, and used boolean later to make a decision. Uses a string value to make a decision.	One or two minor errors in the program logic, including incorrect if statement logic or incorrect math computations.	Multiple errors in the program logic, including incorrect if statement logic and/or incorrect math computations.	Logic and computations accomplish few or none of the program requirements.
User-defined Function Definition and Function Call	Function correctly defined as specified by the requirements, with proper parameter passing and return data type. Function called correctly.	One or two minor errors in function definition, parameter passing, and/or function call.	Many errors in function definition, parameter passing, and/or function call.	Required function was not defined at all.
User-defined Function Logic	Correct internal logic with constants used for fixed values and result correctly returned.	One or two minor errors in function logic.	Task was correctly performed correctly from within the **main** function, OR multiple errors in user-defined function logic.	Task was not performed at all (not performed from a user-defined function, nor from within the **main** function).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Program Output	Displays all of the required output and follows required output format, according to program requirements	Displays all the required output, but there are one or two minor errors in the output format.	Does not display some required output, OR displays all required output, but has multiple errors in the output format.	Displays completely incorrect output, or does not display any output at all.
C++ Constructs / Readability / Miscellaneous	Demonstrates understanding of program structure. Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Program 2: (52 pts)				
Data Storage Points	Constants defined for all fixed values. All variables defined locally, using appropriate data types. Following coding standards naming conventions for all identifiers.	Minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed values OR defined one global variable.	Major errors OR majority of names did not follow naming conventions OR missing multiple constants for fixed values OR defined multiple global variables.	Constants not used, identifier names are not descriptive, and/or there is a multitude of errors.
Program Input	Displays a descriptive prompt for each input and reads the required input correctly. Includes logic so that no unnecessary input is read.	Reads the required input correctly, but does not display an exacting prompt OR reads unnecessary input.	Reads some of the required input incorrectly, OR does not read some required input at all.	Hardcoded values used instead of reading input, OR reading of input fails completely.
Program Logic: if statements and calculations	The program logic accomplishes all program requirements. All math computations compute correct results. All if statements written correctly. Program exits when it should.	Minor errors in the program logic, including incorrect if statement logic, or incorrect math computations.	Many errors in the program logic, including incorrect if statement logic and/or incorrect math computations.	Logic accomplishes few or none of the program requirements.
User-defined Function Definition and Function Call	Function correctly defined as specified by the requirements, with proper parameter passing and return data type. Function called correctly.	One or two minor errors in function definition, parameter passing, and/or function call.	Many errors in function definition, parameter passing, and/or function call.	Required function was not defined at all.
User-defined Function Logic	Correct internal logic with constants used for fixed values and result correctly returned.	One or two minor errors in function logic.	Task was correctly performed correctly from within the **main** function, OR multiple errors in user-defined function logic.	Task was not performed at all (not performed from a user-defined function, nor from within the **main** function).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Output	Displays output correctly, formatted as specified by the requirements.	There is a minor error in the output content or format.	Multiple errors in the output content or format.	Displays completely incorrect data output, or does not display any output.
C++ Constructs / Readability / Miscellaneous	Demonstrates understanding of program structure. Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.

Documentation	Top of program comments include all required components. Top of function comments explain what function will do, and describe all parameters and return values. Inline comments are included as necessary.	Program description is missing some details about the input, output, or processing OR top of function comments vague or missing some details about parameters and return values OR missing other minor necessary comment.	Documentation has many problems with missing, incomplete and/or incorrect items in program description, top of function comments, and/or inline comments.	Only a few (or no) comments in program.

NOTE: Deductions for late submissions will be applied (-2% per day late)
But programs will **not be accepted** if they are submitted **more than 5 days** late (grade will be 0).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law